

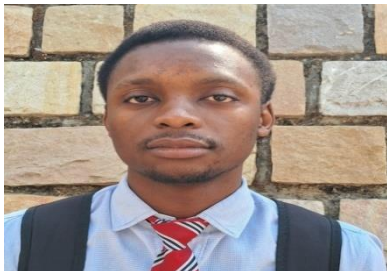


PROJECT AND TEAM INFORMATION

Project Title

LEXICAL ANALYZER GENERATOR

Student / Team Information

Team Name:	CD-VI-T002
Team member 1 (Team Lead)	Reward Kundai Magama: 2024190 Kundaimagam_a05@gmail.com 
Team member 2	Michael Ally Mulungu: 2023452 Vabronoble1@gmail.com 
Team member 3	Aryan Khatri: 2023759 arykha0123@gmail.com 

PROPOSAL DESCRIPTION

Motivation

The Lexical analysis is the first phase of a compiler. It reads the source code and breaks it into small units called tokens. Writing a lexical analyzer manually is time-consuming, error-prone, and hard to update. Although tools like Lex and Flex automate this process, students need to understand how they work. This project builds a simple version of such tools to help students learn how scanner generators function.

State of the Art / Current solution

Some tools that already exist for generating lexical analyzers include Lex, Flex. These tools take regular expressions as input, automatically create scanners, and work very fast. However, they do not clearly show how NFA and DFA are built internally, which makes them difficult for beginners to understand. They are mainly designed for professional use rather than learning. Our project focuses on helping students learn and clearly understand how lexical analyzer generators work.

Project Goals and Milestones

The main goal is to build a Lexical Analyzer Generator and this is going to be achieved by following the steps below:

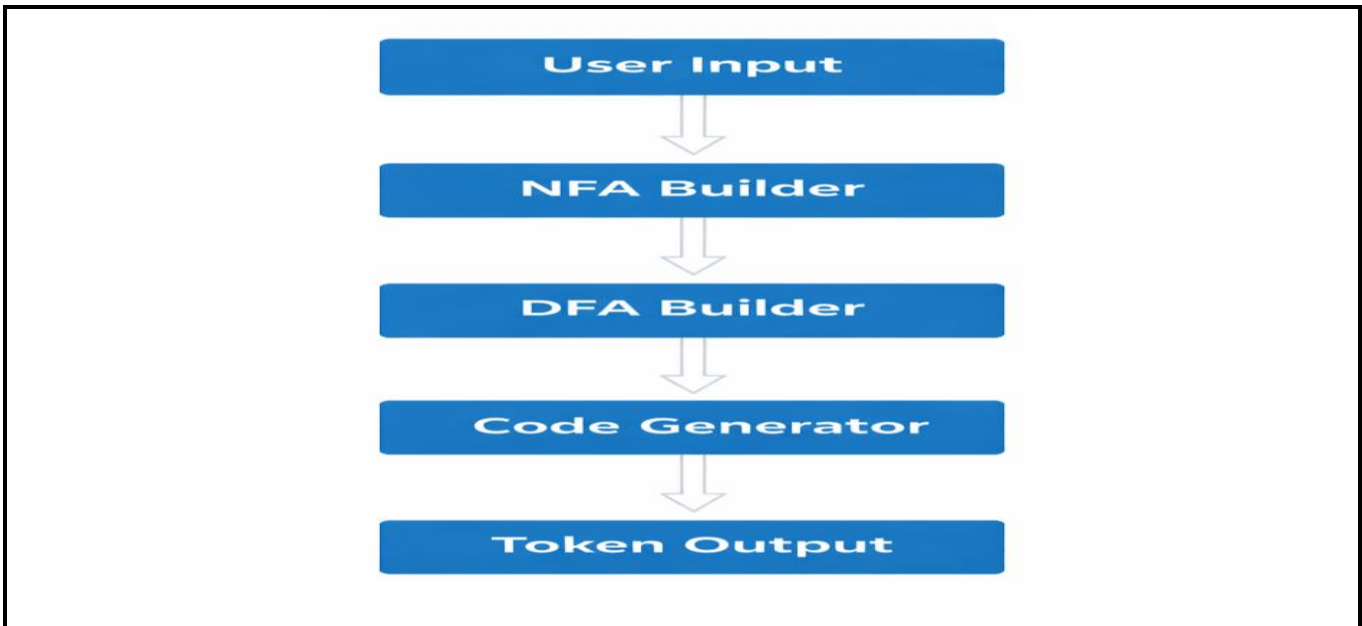
- Step 1:** Read Token Rules.
- Step 2:** Convert Regular Expression to NFA.
- Step 3:** Convert NFA to DFA.
- Step 4:** Generate to C Code.
- Step 5:** Display the DFA and create a simple web interface for user interaction.

Project Approach

We will build the system step by step, starting with the design and moving into implementation and testing.

- Step 1:** Read regular expressions from the input file.
- Step 2:** Convert each regular expression into an NFA.
- Step 3:** Convert the NFA into a DFA.
- Step 4:** Create the DFA transition table.
- Step 5:** Run the generated scanner to produce token output.
- Step 6:** Test each module to make sure the system works correctly.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

By the end of the project, we will deliver:

- A working tool that reads regular expressions.
- Automatic creation of NFA from the given expressions.
- Automatic conversion of NFA into DFA.
- Sample test cases and inputs for testing.

Assumptions

- Input regular expressions follow standard syntax
- Tokens are defined using regular languages only
- The system runs on a standard development environment (GCC compiler)

References

Kumar, S. and Nethravathi, P.S., 2024, April. Revolutionizing compilation: Lexical analysis approaches for parallel multi-core processing. In *2024 International Conference on Inventive Computation Technologies (ICICT)* (pp. 142-149). IEEE.

Pai T, V. and Aithal, P.S., 2020. A systematic literature review of lexical analyzer implementation techniques in compiler design. *International Journal of Applied Engineering and Management Letters (IJAEML)*, 4(2), pp.285-301.

Poojary, M., Penumatsa, H.K., Bhattacharya, R. and Rathore, Y.K., 2025. *Automata Theory and Compiler Design: A Foundational Guide to Automata Theory Concepts and Compiler Design Techniques for Computer Science Learners*/ Dr. Manjula Poojary, Dr. Hima Keerthi Penumatsa, Dr. Riju Bhattacharya, Dr. Yogesh Kumar Rathore/ AG Publishing House/ AGPH Books. AG Publishing House/ AGPH Books.

MISHRA, V.K., VENKATESH, M.K. and PANDURANG, J.A., 2025. *Compiler Design: A Systematic Approach*. Geh press.

Asmitha, M. and Panda, N., 2024, May. Exploring different methods of scanners and parsers. In *2024 International Conference on Advances in Modern Age Technologies for Health and Engineering Science (AMATHE)* (pp. 1-7). IEEE.